

Q1 Annapurna Modern Data Stack

Task-wise Outputs, Explanations and Reasons

This report presents the outputs obtained while completing Question 1 of the Annapurna Modern Data Stack project. Each section explains what was performed, what the output shows, and why the output was produced.

Technologies used: Docker, PostgreSQL, MinIO, DuckDB, Python, CSV and Parquet

Question 1 (A): Project Setup and Data Processing

Task: Set up the project environment and process the source data using the pipeline.

Output:

```
Pipeline execution completed successfully.  
  
Raw rows processed: 1,137,585  
Duplicates removed: 16,661  
Cleaned data and analytical files generated successfully.
```

Explanation: The pipeline read the source data, processed the records and generated the cleaned and transformed datasets. The duplicate count indicates that repeated records were identified and removed before the analytical stage.

Reason for the output: Cleaning is required because duplicate records can inflate sales totals, distort counts and produce incorrect analytical results. Processing the data before loading it into the analytical systems ensures that later queries use a consistent dataset.

Question 1 (B): PostgreSQL Operational Database

Task: Load the required operational and reference data into PostgreSQL and verify the tables.

Output:

```
PostgreSQL database: annapurna  
  
stores : 12 rows  
product_categories : 14 rows  
products : 1,224 rows  
price_revisions : 4,320 rows
```

Explanation: The PostgreSQL database contains the main reference and operational tables used by the project. The row counts confirm that the expected tables were created and populated.

Reason for the output: PostgreSQL is used as the operational database because it supports structured tables, relationships, constraints and SQL queries. Verifying row counts confirms that the data was loaded and that the tables are available for further processing.

Question 1 (C): MinIO Object Storage

Task: Store raw, processed and output files in MinIO as the project's data lake storage layer.

Output:

```
MinIO bucket: annapurna  
Objects stored: 172  
Approximate storage used: 141.7 MiB  
  
Stored content included raw data, processed outputs,  
reports and star-schema files.
```

Explanation: The MinIO bucket contains the files generated during the project. The object count shows that multiple datasets and output folders were uploaded successfully.

Reason for the output: MinIO provides object storage similar to a cloud data lake. It is suitable for storing raw files, CSV files, Parquet files and generated reports independently from the relational database.

Question 1 (D): DuckDB Analytical Processing

Task: Create the DuckDB analytical database and use it for analytical and federated queries.

Output:

```
DuckDB database generated:
annapurna.duckdb

Analytical outputs generated:
daily_sales.csv
monthly_sales.csv
sales_cleaned.parquet
partitioned analytical data
```

Explanation: DuckDB was used to query and transform the processed data for analysis. The generated database and summary files show that the analytical processing stage was completed.

Reason for the output: DuckDB is designed for analytical workloads and can query files such as CSV and Parquet. It allows data analysis without requiring every analytical operation to be performed inside the operational PostgreSQL database.

Question 1 (E): ELT, Star Schema and Parquet Outputs

Task: Transform the data into analytical structures and generate curated datasets in Parquet format.

Output:

```
Generated files included:

stores.csv
products.csv
product_categories.csv
price_revisions.csv
sales_cleaned.parquet
partitioned/
star_schema/
```

Explanation: The project generated separate reference datasets, a cleaned sales dataset, partitioned data and star-schema outputs. These files represent the curated layer used for analysis.

Reason for the output: A star schema separates measurable business events from descriptive dimensions, making analytical queries easier to understand. Parquet was used because it is columnar, supports efficient analytical reads and is suitable for data-lake storage.

Question 1 (F): Finance Reconciliation and Final Submission

Task: Calculate monthly sales values, compare the results with the available finance reference and prepare the final submission files.

Output:

```
Finance reconciliation file generated:  
reports/finance_reconciliation.csv
```

```
Final submission folder contained:  
annapurna.duckdb  
daily_sales.csv  
finance_reconciliation.csv  
monthly_sales.csv  
price_revisions.csv  
products.csv  
product_categories.csv  
sales_cleaned.parquet  
stores.csv  
partitioned/  
star_schema/
```

Explanation: The reconciliation file records the calculated monthly values and provides a way to compare analytical results with the finance reference data. The final submission folder contains the database, cleaned data, summaries and schema outputs required for evaluation.

Reason for the output: Reconciliation is necessary to check whether the analytical calculations are consistent with the available finance information. Preparing a separate final submission folder also makes the project easier to review, share and upload to GitHub.

Overall Conclusion

Question 1 was completed by building a local data stack consisting of PostgreSQL for structured operational data, MinIO for object storage and DuckDB for analytical processing. The source data was cleaned, transformed and stored in reusable CSV and Parquet outputs. The generated reconciliation file and final submission folder provide evidence of the completed processing and make the project ready for submission and GitHub version control.

Note: The outputs in this report are based on the results generated during the project execution. The report focuses on showing the output and explaining the reason behind each result.